Documentos

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

Documentos

https://github.com/mikel-egana-aranguren/ABD

CC BY

Introducción a MongoDB

Administración de Bases de Datos

Departamento de Lenguajes y Sistemas Informáticos

Contenido

- Introducción a NoSQL
- MongoDB
- Instalación
- Estructura de los datos
- Comandos CRUD
- Relaciones
- Esquemas

Introducción

- En ciertos entornos, las BBDD SQL pueden ser demasiado rígidas.
- Ejemplos:
- Almacenar correos electrónicos o mensajes de texto.
- La longitud del texto puede variar mucho.
- Los mensajes pueden tener ficheros adjuntos.
- Gestionar mensajes de log de servidores
- Cada mensaje de log puede contener campos diferentes
- La longitud de cada campo puede variar

Introducción

- Los sistemas NoSQL (Not Only SQL) ofrecen una alternativa a las BBDD SQL.
- Características relevantes:
- No requieren esquema
- No es necesario definir explícitamente el formato de los datos
- Facilidad para escalabilidad horizontal
- Preparados para crecer añadiendo más nodos/VMs
- Particionado horizontal (sharding)
- Los datos se distribuyen entre los nodos disponibles
- Favorece el acceso concurrente

Introducción

- Tipos de SGBDs en entornos empresariales
- Según “2021 State of Database DevOps Survey” de Redgate1

4

1 a 100

empleados

101 a 1000

empleados

Más de 1000 empleados

1: Redgate “2021 State of Database Devops”: https://www.red-gate.com/solutions/database-devops/report-2021

Introducción

- Categorías de sistemas NoSQL:
- Basados en documentos
- Organizan los datos utilizando formatos como JSON
- P.e. MongoDB
- Clave-valor
- Cada dato tiene una clave asociada
- P.e. Redis

5

Introducción

1: Row vs Column Oriented Databases: https://dataschool.com/data-modeling-101/row-vs-column-oriented-databases/

- Categorías de sistemas NoSQL:
- Basados en columnas
- Organizan los datos en filas1
- P.e. Apache Druid
- Basados en grafos
- Los datos se representan como grafos
- Se recorren utilizando expresiones de rutas.
- P.e. Neo4J

Introducción

- En este tema utilizaremos **MongoDB**
- https://www.mongodb.com/
- BD NoSQL orientada a documentos
- Creada en 2009
- Última versión: 6.0
- Su nombre viene de la palabra Humongous ("*gigantesco*")
- Nació de la frustración de usar SQL1

1: When to Use a NoSQL Database: https://www.mongodb.com/nosql-explained/when-to-use-nosql

Introducción

- Ranking de popularidad de SGBDs
- Según DB-Engines.com: db-engines.com/en/ranking_trend
- Fecha: 18 de abril de 2023

MongoDB

Ecosistema de productos MongoDB:

- MongoDB Community Edition
- Versión libre de la BD
- En este tema usaremos esta versión
- MongoDB Enterprise
- Versión comercial de la DB
- Atlas
- Servicio de BD en la nube
- Herramientas adicionales
- P.e. Compass (GUI para MongoDB)

MongoDB: Instalación

- Instalación de Community Edition en Ubuntu 22.04:
- https://www.mongodb.com/docs/manual/tutorial/install- mongodb-on-ubuntu/#install-mongodb-community-edition
- Pasos:
- Importar la clave pública del repositorio en *apt*.
- Crear un fichero de listado para MongoDB.
- Recargar el listado de paquetes disponibles en *apt*.
- Instalar usando *apt*.

MongoDB: Instalación

- Comprobar el estado de MongoDB:
- Iniciar/detener MongoDB manualmente:
- Utilizar *start* para iniciar o *stop* para detener
- Iniciar MongoDB con el arranque del sistema:
- Abrir una consola de MongoDB:

```
sudo service mongod status

sudo service mongod start|stop

sudo systemctl enable mongod.service

mongosh
```

MongoDB

- Estructura de los datos
- Ejemplo con la BD “Tienda”

Tienda

Clientes

Pedidos

Bases de datos

Colecciones

Documentos

{nombre: “Unai”, DNI: “12345678”}

{nombre: “Jon”}

{ … }

{ … }

MongoDB

- Estructura de los datos
- Un documento se organiza en formato JSON1
- JavaScript Object Notation
- Cada dato se almacena como clave:valor
- Ejemplo para un “cliente”:
  {

  13

  1: Internamente se almacena en formato Binary JSON (BSON)

  nombre: “Unai”, DNI: “12345678A”,

  direccion:

  {

  ciudad: “Bilbao”

  }

MongoDB

- Estructura de los datos
- Equivalencia aproximada con el sistema relacional
- Diferencias principales:
- No todos los documentos tienen por qué tener las mismas claves
- No es necesario definir relaciones explícitas entre documentos

Sistema relacional	Sistema basado en documentos
Tabla	Colección
Fila	Documento
Columna/Campo	Clave

MongoDB: Comandos

- Limpiar la pantalla
- Mostrar BBDD existentes
- Mostrar BBDD en uso
- Cambiar a una BBDD
- cls
- show dbs
- db
- use <nombre-BBDD>

MongoDB: Comandos

- Creación de BBDD y colecciones
- Por defecto, MongoDB funciona en modo de creación implícita
- Si se accede a un objeto que no existe, se crea automáticamente
- Ejemplo:
- Crear la BBDD “tienda” y la colección “clientes”
- Es necesario crear un primer documento en “clientes”
- use tienda
- db.clientes.insertOne({ “nombre”: “Unai” })

Nombre de la colección

Método para insertar 1 documento

Documento con datos de un cliente

MongoDB: Comandos

- Creación de BBDD y colecciones
- En MongoDB, cada documento tiene un identificador único
- Se asigna automáticamente en su creación
- Aunque también podemos asignar uno de forma manual
- Si la creación ha sido correcta, se mostrará algo similar a:
- donde:
- *Acknowledged:true* indica que no ha habido fallos
- *insertedId*: indica el identificador del documento

```
{
acknowledged: true,
insertedId: ObjectId("626169bbae7b857469c4514b")
}
```

MongoDB: Comandos

- Creación de BBDD y colecciones
- Insertar múltiples documentos en una colección
- donde <array-docs> es un Array de documentos.
- Ejemplo:
- Insertar múltiples documentos en la colección “clientes”:
- db.<nombre-colección>.insertMany(<array-docs>)
- db.clientes.insertMany([
  {nombre: "Amaia” , DNI: 33344},

  {nombre: "Ainhoa", DNI: 33455}

  ])

MongoDB: Comandos

- Mostrar el contenido de una colección
- Mostrar todos los documentos:
- Mostrar los documentos que encajen con un patrón:
- Encontrar el 1er documento que encaje con un patrón:
- Sin parámetros mostraría el 1er documento de la colección
- db.<nombre-colección>.find()
- db.<nombre-colección>.find(<patrón>)
- db.<nombre-colección>.findOne(<patrón>)

MongoDB: Comandos

- Patrones en las operaciones CRUD
- Para filtrar los documentos a los que afecta cada operación CRUD se puede proveer un patrón
- El patrón es un objeto JSON que indica los campos a buscar.
- Ejemplo: Buscar en la colección “clientes” los clientes con nombre “Unai”:
- Ejemplo: Buscar en “clientes” los clientes con nombre “Unai” y DNI 12345:
- db.clientes.find({ nombre: "Unai" })
- db.clientes.find({ nombre: "Unai", DNI: 12345 })

MongoDB: Comandos

- Patrones en las operaciones CRUD
- Para filtrar los documentos a los que afecta cada operación CRUD se puede proveer un patrón
- Se pueden utilizar operadores de comparación
- Los más comunes: eq (=), ne (!=), gt (>), gte (>=), lt (<), lte (<=)
- Ejemplo: Buscar en la colección “clientes” los DNIs mayores que 30000:
- Se pueden utilizar expresiones regulares
- Ejemplo: Buscar clientes cuyo nombre empiece por M
- db.clientes.find({ DNI: { $gt: 30000 } })
- db.clientes.find({ nombre: /M./ })

MongoDB: Comandos

- Mostrar el contenido de una colección
- El 2º parámetro del comando *find* permite indicar los campos a mostrar para cada objeto.
- Evitar leer datos de más, útil en BBDD grandes.
- Para cada campo, utilizar 1 como valor para mostrarlo o 0 para omitirlo.
- Si se utiliza el 2º parámetro, por defecto todos los campos se omiten.
- Ejemplo:
- Recuperar sólo el DNI de los documentos en la colección “clientes”

  { campoAMostrar: 1, campoAOcultar: 0}

- db.clientes.find({} , {DNI: 1, _id: 0})

MongoDB: Comandos

- Mostrar el contenido de una colección
- Comparación con SQL
- Ejemplo: Recuperar el nombre del cliente con nº de DNI 33344
- En SQL:
- En MongoDB:
- Más comparaciones de SQL con MongoDB:
- https://www.mongodb.com/docs/manual/reference/sql-comparison/
- db.clientes.find({DNI: 33344}, {nombre:1, _id:0})

  mysql> SELECT nombre FROM clientes WHERE DNI=3344;

MongoDB: Comandos

- Eliminar documentos
- Eliminar el 1er documento que encaje con <patrón>
- Eliminar los documentos que encajen con <patrón>
- Eliminar todos los documentos de una colección
- db.<nombre-colección>.deleteOne(<patrón>)
- db.<nombre-colección>.deleteMany(<patrón>)
- db.<nombre-colección>.deleteMany({})

MongoDB: Comandos

- Modificar documentos
- Modificar documentos en una colección:
- donde:
- <patron-busq>: Patrón para buscar los documentos a modificar
- <cambios>: Objeton JSON con los campos a modificar
- Ejemplo:
- Para los clientes cuyo DNI sea 22233, actualizar su nombre a “Nagore”:
- db.<nombre-colección>.updateMany(<patrón-busq>, {$set: <cambios>})
- db.clientes.updateMany({ DNI: 22233 }, {$set:{ nombre: "Nagore" } })

MongoDB: Comandos

- Modificar documentos
- Modificar un documento de una colección:
- donde <patrón-busq> y <cambios> funcionan como se indica en la diapositiva anterior.
- Reemplazar un documento por otro:
- donde:
- <patrón-busq>: Patrón de búsqueda
- <nuevo-doc>: Documento que reemplazará a los documentos que encajen con <patrón-busq>
- db.<nombre-colección>.replaceOne(<patrón-busq>, <nuevo-doc>)
- db.<nombre-colección>.updateOne(<patrón-busq>, {$set: <cambios>})

MongoDB: Comandos

- Mostrar colecciones en una BD:
- Eliminar una colección completa:
- Eliminar una BD:
- db.<nombre-colección>.drop()
- db.dropDatabase()
- show collections

Ejercicio 1

- Crear una BBDD llamada “cine” que contenga las siguientes colecciones y documentos: Colección “peliculas” Colección “actores”
- *Continúa en la siguiente diapositiva*

nombre	año
Her	2013
Joker	2019
Judy	2019

nombre	edad
Joaquín Phoenix	12
Renee Zellweger	53
Keanu Reeves	58

Ejercicio 1

- Modificar la edad de Joaquín Phoenix a 48 años.
- Eliminar a Keanu Reeves de la colección “actores”.
- Añadir el siguiente documento a la colección “películas”:
- nombre: “Avatar”
- año: 2009
- nota: 7.2
- Mostrar únicamente los nombres de las películas estrenadas a partir de 2015.

Relaciones

- Un documento puede contener:
  Documentos embebidos

  nombre: “Unai”, DNI: “12345678A”,

  direccion: { ciudad: “Bilbao”, CP: 48013,

  ubicacion: {

  calle: “R. Moreno”, piso: 3,

  despacho: 17

  }

  }

  { {

  nombre: “Unai”, DNI: “12345678A”,

  telefonos: [

Relaciones

- En un entorno real, es habitual tener:
- Varias colecciones
- Cada una representa una entidad del contexto.
- Relaciones entre los datos de diferentes entidades.
- P.e. “clientes”, “productos” y ”pedidos”
- MongoDB no proporciona una técnica concreta para definir relaciones entre colecciones.
- Las debemos definir nosotros.

Relaciones

- En MongoDB, hay 2 formas de relacionar información de diferentes entidades:
- Documentos embebidos

Colección "clientes"

{

nombre: "Unai", DNI: "12345678A",

direccion:

{

ciudad: "Bilbao", CP: 48013,

...

}

Características:

- Adecuado para datos que no se solapan/repiten

Relaciones

- En MongoDB, hay 2 formas de relacionar información de diferentes entidades:
  2) Utilizar campos concretos como referencia

  Colección "clientes"

  {

  nombre: "Unai", DNI: "12345678A",

  direcciones: "001",

  ...

  Colección "direcciones"

  {

  _id: "001", ciudad: "Bilbao", CP: 48013,

  ...

- Adecuado para datos que se referencian en diferentes colecciones

Relaciones

- Se pueden utilizar referencias para resolver 2 escenarios posibles:
- Relación entre los datos 1 a N:
- P.e. un cliente puede tener múltiples direcciones, una dirección pertenece sólo a 1 cliente.

Colección "clientes"

{

nombre: "Unai", DNI: "12345678A",

direcciones: ["001", "002"],

...

Colección "direcciones"

{

_id: "001", ciudad: "Bilbao", CP: 48013,

...

Relaciones

- Se pueden utilizar referencias para resolver 2 escenarios posibles:
- Relación entre los datos N a N:
- P.e. un cliente puede comprar múltiples productos,

un producto puede ser comprado por múltiples clientes.

Colección "clientes"

{

_id = "cli-01" nombre: "Unai", DNI: "12345678A",

...

Colección "producto"

{

_id: "pro-02", marca: "Apple", modelo: "iPhone 14"

Relaciones

- ¿Cuando utilizar cada uno?

  Documentos embebidos

  Referencias

- Datos que estén fuertemente relacionados y que no estén duplicados.
- Relaciones 1 a 1.
- P.e. un cliente tendrá sólo una dirección asociada.
- Relaciones 1 a N donde no haya duplicidades.
- Datos de entidades independientes pero relacionadas.
- Relaciones 1 a N.
- Relaciones N a N.

Relaciones

- Se pueden combinar los datos de diferentes colecciones mediante una operación de agregación
- Similar a la operación Join relacional.
- Ejemplo:
- Crear una colección que contenga los datos de los pedidos combinados con los datos de los clientes

Colección "pedidos"

{ _id: "ped-001", cliente: "cli-01", productos: ["pro-01"]

}

Colección "clientes"

{ _id = "cli-01" nombre: "Unai", DNI: 45823150C }

Combinación

{ _id: "ped-001", cliente: "cli-01", productos: ["pro-01"] datosCliente: {

_id = "cli-01" nombre: "Unai", DNI: 45823150C

Relaciones

- La operación de agregación y el operador $lookup para combinar colecciones.
- Ejemplo: Crear una colección que contenga los datos de los pedidos combinados con los datos de los clientes
- db.pedidos.aggregate([
  {

  $lookup: {

  from: "clientes", localField: "cliente", foreignField: "_id", as: "datosClientes"}

  }

  ])

  Colección a combinar

  Colección “clientes”

  { _id = “cli-01” nombre: “Unai”, DNI: 45823150C }

  Colección “pedidos”

Ejercicio 2

- *Utilizando la BD "cine" creada en el ejercicio 1:*
- Añadir un campo "premios" a los documentos de "actores" con la siguiente información:
- Crear un listado de documentos
- *Continúa en la siguiente diapositiva*

actor/actriz	nombre	año
Renée Zellweger	Oscar a la mejor actriz	2020
Renée Zellweger	Globo de oro a la mejor actriz	2020
Joaquin Phoenix	Oscar al mejor actor	2020

Ejercicio 2

- Añadir un campo “participa” a los documentos de “actores” donde se referencie a las películas en las que ha participado:
- Crear un listado de “ObjectId”s
- Mostrar los datos de Joaquin y Renée, incluyendo todos los datos de las películas en las que han participado

actor/actriz	**película**
Joaquin Phoenix	Her
Joaquin Phoenix	Joker
Renée Zellweger	Judy

Esquemas

44

- Es posible gestionar BBDD en MongoDB sin definir ningún tipo de estructura.
- *pero…*
- En algunas situaciones puede que queramos controlar los datos de forma automática
- P.e. que todos los documento de una colección “productos” tienen un campo numérico “precio”.

Esquemas

- Posibles enfoques de diseño1:

  45

  {

  nombre: “Libro”, precio: 5.99

  }

  {

  nombre: “Botella”, precio: 2.55, disponible: true

  }

  {

  nombre: “Libro”, precio: 5.99

  }

Esquemas

- Se puede definir la estructura que deben cumplir los documentos de una colección.
- Un esquema verifica cada documento insertado en una colección y genera un aviso/error si es incorrecto.

Colección

Esquema de validación

Insertar documento

¿Documento correcto?

Generar aviso/error

Nueva inserción

Si

No

Esquemas

- Los esquemas de validación tienen 2 opciones que controlan su comportamiento:
- validationLevel
- Controla cómo de estricta es la validación. 2 opciones:
- **strict** (valor por defecto): Se comprueba toda inserción y modificación.
- **moderate**: Las modificaciones a documentos ya existentes no se comprueban.
- validationAction
- Indica qué hacer cuando un documento no cumple el esquema. 2 opciones:
- **error** (valor por defecto): Se emite un error y se impide la inserción.
- **warn**: Se escribe un aviso en el log de MongoDB y se permite la inserción.

Esquemas

- Para incluir un esquema, se debe crear una colección de forma explícita con createCollection:
- El esquema se define en el 2º parámetro con un campo “validator”.
- db.createCollection("marcas", { validator: {
  $jsonSchema: { bsonType: "object",

  required: ["nombre", "presupuesto"], properties: {

  nombre: { bsonType: "string", description: "nombre de la marca"}, presupuesto: { bsonType: "int", minimum: 2000 }},

  },

  validationLevel: 'moderate', validationAction: 'error'

  })

  Opcionales

  Listado de campos de la colección

Esquemas

- Mostrar el esquema de una colección:
- Modificar el esquema de una colección:
- También sirve para añadir un esquema a una colección existente.
- Si validationAction es “warn”, el resultado se escribe en el log de MongoDB.
- Por defecto, la ubicación del fichero log es /var/log/mongodb/mongod.log
- db.getCollectionInfos({name: "marcas"})
- db.runCommand({collMod: "marcas", validator: {
  $jsonSchema: {

  …

Esquemas: Tipos de datos

Nombre	Descripción	Ejemplo
string	Texto plano	“Unai”
Boolean	Valor booleano	true
int	Número entero (int32)	55
NumberLong	Número entero grande (int64)	1000000000
NumberDecimal	Números decimales	12.99
ObjectId	Identificador único	ObjectId(“74121...”)
ISODate	Fecha en formato AAAA-MM-DD	2022-01-09
Timestamp	Fecha en formato Unix	11348822
EmbeddedDocument	Documento embebido	{ “a”: { ... } }
Array	Listado de elementos	{ “b”: [...] }

Esquemas: Tipos de datos

- Fechas:
- Nuevo fecha tipo ISODate
- Recoger la fecha actual del sistema:
- Indicar una fecha concreta:
- Nueva fecha tipo Unix:

```
{ miFecha: new Date() }

{ miMarcaDeTiempo: new Timestamp() }

{ miFecha: new Date("1998-08-08") }
```

Ejercicio 3

- *Utilizando la BD "cine" de los ejercicios 1 y 2:*
- Crear una colección "salas" con el siguiente esquema:
- Campos requeridos
- nombre (tipo: string)
- poblacion (tipo: string)
- capacidad (tipo: int, como mínimo debe ser 150)
- Nivel de validación: estricto
- Acción de validación: aviso (warning)
- *Continúa en la siguiente diapositiva*

Ejercicio 3

- Introducir los siguientes documentos en ”salas”:
- Verificar que se ha escrito un aviso en el log debido a la inserción de la sala “Autocine”.

nombre	población	capacidad
Yelmo Megapark	Barakaldo	500
Autocine	Getxo	100
Zubiarte	Bilbao	250

Bibliografía

55

- R. Elmasri, S. B. Navathe. “Fundamentals of Database Systems”, 7th edition, Pearson, 2017
- M. Schwarzmüller. “MongoDB - The Complete Developer's Guide”, Udemy, 2022.
- https://www.udemy.com/course/mongodb-the-complete- developers-guide
- MongoDB Documentation, 2022.
- https://www.mongodb.com/docs

Optimización en MongoDB

Administración de Bases de Datos

Departamento de Lenguajes y Sistemas Informáticos

Contenido

- Índices
- Explain
- Colecciones limitadas

Índices

- Al igual que en los sistemas relacionales, en MongoDB se pueden añadir índices para agilizar consultas.
- Se definen para uno o varios campos de una colección.
- Tienen las mismas desventajas que en los sistemas relacionales:
- Ocupan espacio en disco
- Hacen que las operaciones de actualización se ralenticen

Índices

- Añadir índice al campo de una colección:
- donde:
- *campo*: Campo al que queremos añadir el índice
- *ORDEN:* 1 para que el índice sea ascendente, -1 para que sea descendente.
- En general, el orden del índice no afecta en exceso.
- MongoDB es capaz de recorrer un índice en ambas direcciones
- Ejemplo:
- db.<colección>.createIndex({ campo: ORDEN })
- db.clientes.createIndex({ DNI: 1 })

Índices

- Se pueden crear índices compuestos que relacionen diferentes campos de una colección
- Al igual que en los sistemas relacionales
- Ejemplo:
- Índice sobre el campo “edad” junto con el campo “DNI”:
- Datos de ejemplo a la derecha.
- db.clientes.createIndex(
  {

  edad: 1, DNI: 1

  })

edad	DNI
20	1112223
20	1155566
20	2227744

Índices

- Se puede especificar que el índice sólo recoja valores concretos de un campo.
- Útil si sabemos que un % elevado de nuestras consultas van a un rango determinado de valores.
- El índice creado ocupará menos espacio en disco.
- Se utiliza el 2º parámetro de createIndex
- Ejemplo: indexar sólo DNIs con valor igual o mayor que 50000000.
- db.clientes.createIndex(
  { DNI: 1 },

  {

  partialFilterExpression: { DNI: { $gte: 50000000 }

  }

  })

  Definición del rango

Índices

- MongoDB proporciona un tipo concreto de índice para campos de texto.
- Segmenta un campo de texto en palabras, elimina palabras vacías1 e indexa en base a las palabras restantes.
- Ejemplo:
- Para la siguiente cadena de texto:
- Se indexarían las siguientes palabras:

6

1Definición de “palabra vacía”: https://es.wikipedia.org/wiki/Palabra_vac%C3%ADa

“This product is a must-buy for all fans of modern fiction”

product

must

fiction

modern

Índices

- MongoDB proporciona un tipo concreto de índice para campos de texto.
- Crear índices de texto tiene un alto coste computacional y MongoDB sólo permite tener uno por cada colección
- Crear un índice de texto sobre un campo:
- Crear un índice de texto sobre varios campos:
- db.<colección>.createIndex({ campo1: “text” })
- db.<colección>.createIndex({ campo1: “text”, campo2: “text” })

Índices

- MongoDB proporciona un tipo concreto de índice para campos de texto.
- Para buscar en una colección que contenga un índice de texto, se utilizan las palabras clave $text y $search.
- Busca documentos que contengan “libro”
- Busca documentos que contengan “libro” o “rojo”:
- Busca documentos que contengan “libro rojo”:
- No es necesario indicar el campo sobre el que se ha establecido el índice.
- db.<colección>.find({ $text: { $search: "libro"} })
- db.<colección>.find({ $text: { $search: "libro rojo"} })
- db.<colección>.find({ $text: { $search: "\"libro rojo\""} })

Índices

- Mostrar los índices en una colección:
- Ejemplo: índices en la colección “clientes”
- Para eliminar un índice, se utiliza la función dropIndex con los mismos parámetros utilizados para crearlo.
- Ejemplo: Eliminar un índice ascendente sobre el campo DNI.
- Para eliminar índices de texto hay que utilizar el nombre del índice.
- Se puede obtener con el comando getIndexes.
- db.clientes.dropIndex({ DNI: 1 })
- db.clientes.getIndexes()
- db.<colección>.dropIndex({“nombre-del-índice”})

Explain

- La función Explain proporciona información sobre tiempos y plan de ejecución de una consulta.
- Equivalente al comando Explain Analyze en sistemas relacionales.
- Se ejecuta como función de la colección:
- Ejemplo:
- db.<colección>.explain().<comandoAAnalizar>
- db.clientes.explain().find({ DNI: 12345678})

Explain

- El comando Explain recibe como parámetro un *string* que determina el nivel de detalle que muestra:
- Ejemplo:
- Valores posibles e información mostrada:
- db.clientes.explain("executionStats").find({ DNI: 12345678})

	"queryPlanner"	**"executionStats"**	**"allPlansExecution"**
Estadísticas de la ejecución	✓	✓	✓
Detalles del plan ejecutado		✓	✓
Planes descartados		✓	
Proceso de decisión			✓

Importar datos

- Mongoimport permite importar datos en formato JSON
- donde:
- <ruta-fichero>: Ruta y nombre del fichero JSON con datos
- <BD>: BD de MongoDB en la que importar los datos
- <colección>: Colección de <BD> en la que importar los datos
- Ejemplo:
- Importar el contenido del fichero misDatos.json en la colección proveedores de la BD tienda

```
$ mongoimport <ruta-fichero> -d <BD> -c <colección>

$ mongoimport misDatos.json -d tienda -c proveedores
```

Ejercicio 1

- Importar el siguiente dataset en MongoDB:
- Repositorio: https://github.com/mcampo2/mongodb-sample-databases/tree/master/sample_airbnb
- Raw: https://github.com/mcampo2/mongodb-sample-databases/raw/master/sample_airbnb/listingsAndReviews.json
- Dataset de muestra de MongoDB
- Contiene datos y comentarios para 5555 alojamientos de AirBnB
- *Continúa en la siguiente diapositiva*

Ejercicio 1

- Realizar las siguientes consultas:
- Obtener cuántos alojamientos tienen un *rating* superior a 90.
- Campo review_scores.review_scores_rating
- Obtener cuántos alojamientos tienen uno (o más comentarios) donde se les califique como “awesome”
- Campo reviews.comments
- Usar {$regex: "awesome"}
- Obtener el tiempo de ejecución de las consultas
- Crear índices en los campos afectados por las consultas
- Probar diferentes tipos de índices y medir cuánto se reducen los tiempos de ejecución

Colecciones limitadas

- Son colecciones con un límite máximo de documentos
- Cuando se llegue al límite, la inserción de un nuevo documento provoca que se elimine el más antiguo.
- Ejemplo: Colección limitada a 3 documentos

{ nombre: “A”}

{ nombre: “A”}

{ nombre: “B”}

Inserción de “A”

{ nombre: “C”}

{ nombre: “B”}

{ nombre: “A”}

{ nombre: “D”}

{ nombre: “C”}

Colecciones limitadas

- Útiles cuando no interese conservar los datos más antiguos y sea necesario ahorrar espacio en disco
- Posibles escenarios de uso:
- Monitorización de servidores que generen muchos logs
- Caché de documentos recientes
- Crear una colección limitada
- donde:
- <max-tam>: Tamaño máximo en Bytes que puede ocupar la colección.
- <max-docs>: Número máximo de documentos permitidos en la colección.
- db.createCollection("nombre",{capped: true, size: <max-tam>, max: <max-docs}})

Ejercicio 2

- Crear una BD “masrecientes” y una colección limitada “llamadas” con un límite máximo de 3 documentos.
- Introducir los siguientes documentos:
- Verificar que la inserción del último documento provoca que se elimine el primero.

contacto	número
Mikel	944123456
Ainhoa	946015588
Unai	612345667
Amaia	612398765

Bibliografía

- R. Elmasri, S. B. Navathe. “Fundamentals of Database Systems”, 7th edition, Pearson, 2017
- M. Schwarzmüller. “MongoDB - The Complete Developer's Guide”, Udemy, 2022.
- https://www.udemy.com/course/mongodb-the-complete-developers-guide
- MongoDB Documentation, 2022.
- https://www.mongodb.com/docs

Administración MongoDB

Administración de Bases de Datos

Departamento de Lenguajes y Sistemas Informáticos

Contenido

- Usuarios y roles
- Copias de seguridad
- Sharding

Usuarios y roles

- Hasta ahora hemos utilizado MongoDB de forma local con permisos de administrador.
- *pero…*
- En un entorno real necesitamos gestionar la actividad mediante usuarios y permisos.

Usuarios y roles

- En MongoDB, los usuarios y permisos se gestionan a través de roles.
- Un rol es un conjunto de permisos para realizar acciones en un recurso, donde:
- Un recurso es, p.e., una colección de una BD.
- Una acción es, p.e., insertar documentos.

Usuarios y roles

- Ejemplo:
- Posibles roles para una base de datos “Tienda”.

Administrador

Analista de datos

Desarollador / App

- Gestiona la configuración
- Gestiona usuarios
- No tiene por qué leer o insertar datos en las colecciones
- Inserta, modifica y lee datos
- No debe modificar la configuración o permisos de la BD
- Lee datos
- No debe modificar la configuración o permisos de la BD
- No debe insertar o modificar datos

Usuarios y roles

- MongoDB tiene roles predefinidos.
- Listado de los más relevantes:
- Listado completo en 1

5

Categoría	Rol	Permisos
Usuario de 1 BD	read	Leer todas las colecciones
	readWrite	Leer y modificar todas las colecciones
Administrador de 1 BD	userAdmin	Gestionar usuarios
	dbOwner	Equivale a userAdmin + readWrite
Administrador de todas las BBDD	readAnyDatabase	Leer cualquier colección
	readWriteAnyDatabase	Leer y modificar cualquier colección
	root	Control total

Usuarios y roles: Activación

- Por defecto, MongoDB:
- No tiene ningún usuario creado
- No tiene activado el control de usuarios
- Sólo permite conexiones locales
- Antes de activar el control de usuarios, es necesario crear un usuario “root”:
- Ejemplo:
- use admin
- db.createUser({
  ... user: "unai",

  ... pwd: passwordPrompt(),

  ... roles: ["root"]

  ... })

  Asignar el rol “root”

  al usuario “unai”

Usuarios y roles: Activación

- Tras crear un usuario “root”, activar el control de usuarios:
- Editar el fichero de configuración de MongoDB:
- Por defecto, /etc/mongod.conf

2) Reiniciar el servicio de MongoDB para aplicar los cambios

...

security: authorization: enabled

...

enabled para activar el control de usuarios

Usuarios y roles: Activación

- El usuario con rol ”root” se crea en la BD “admin”
- Un usuario se crea vinculado a una BD concreta
- El usuario se autentica contra esa BD
- No impide que el usuario acceda a otras BBDD
- Pero tiene que tener permisos para ello
- Conectarse con autenticación a MongoDB:
- Desde la consola Linux:
  $ mongosh –u <usuario> -p --authenticationDatabase <DB>

  Indicar la BD

  vinculada al usuario

Usuarios y roles

- Crear nuevos usuarios con el comando createUser
- Ejemplo: crear el usuario “mikel” vinculado a la BD “tienda” con diferentes roles.
- La contraseña se podría escribir en texto plano en el campo “pwd”, pero no es recomendable por seguridad.
- El campo “roles” contiene un listado de los roles a asignar al usuario, tanto para su BD vinculada, como para otras existentes.
- use tienda
- db.createUser({ user: "mikel",
  pwd: passwordPrompt(), roles: [

  { role: "readWrite", db: "tienda" },

  { role: "read", db: "edificios" }

  ]

  })

Usuarios y roles

- Modificar un usuario con el comando updateUser:
- donde:
- <nombre>: Nombre del usuario en un string, p.e. “mikel”
- <cambios>: Documento indicando los cambios a realizar, p.e.:
- Asignar la contraseña a “Pass456” y el rol a “read” en la BD que está vinculado.
- Asignar el rol “readWrite” en la BD “almacen”.

  { pwd: “Pass456”, roles: [“read”]}

  { roles: [

  { db: ”almacen”, role: “readWrite”}

  ]}

- db.updateUser(<nombre>, <cambios>)

Usuarios y roles

- Visualizar los roles de un usuario, utilizar el comando getUser.
- Es necesario estar usando la BD a la que está vinculado el usuario.
- Eliminar un usuario con el comando dropUser.
- Es necesario estar usando la BD a la que está vinculado el usuario.
- db.getUser(<nombre>)
- db.dropUser(<nombre>)

Usuarios y roles

- Listado de acciones posibles: https://www.mongodb.com/docs/manual/reference/privilege-actions/
- Se pueden crear roles personalizados
- Ejemplo: crear un rol que permita exclusivamente leer y actualizar documentos de la colección “productos” en la BD “tienda”.
- db.createRole(
  {

  role: "dbTiendaLeerCambiarProductos", privileges: [

  { resource:

  {

  db: "tienda", collection: "productos" }, actions: ["find", "update"]

  }

  ],

  roles: []

Ejercicio 1

- Pre-requisito:
- Haber creado un usuario “root” y activado el control de usuarios.
- Como usuario “root”:
- Crear una BD “libreria” que contenga una colección “libros”.
- Insertar 3 documentos en “libros”.
- Crear un usuario “koldo” con permisos de lectura en “libros”.
- Conectarse a MongoDB como “koldo” y:
- Verificar puede leer el contenido de “libros”.
- Verificar que no puede insertar documentos en “libros”.

Copias de seguridad

- Realizar copias de seguridad periódicas es una tarea fundamental de administración.
- Hay 2 formas principales en MongoDB:
- Utilizando sus propias plataformas en la nube (p.e. Atlas).
- Requiere contratar el servicio.
- De forma manual
- Lo veremos en este tema.

Copias de seguridad

- mongodump realiza una copia de una BD en ficheros BSON.
- donde:
- <usuario>: Usuario para autenticarse a MongoDB y realizar la copia
- <BD-aut>: BD vinculada al usuario para autenticación
- <BD>: BD que se quiere copiar. Si este parámetro se omite, se copian todas.
- Se crea una carpeta *dump* con los ficheros BSON.
- Es necesario que el usuario tenga permisos para leer las BDs que se quieran copiar.

```
$ mongodump -u <usuario> --authenticationDatabase <BD-aut> -d <BD>
```

Copias de seguridad

- mongorestore restaura una copia hecha con mongodump.
- donde:
- <usuario>: Usuario para autenticarse a MongoDB y restaurar la copia
- <BD-aut>: BD vinculada al usuario para autenticación
- <BD>: BD en la que restaurar la copia. Opcional.
- <carpeta-dump>: Ruta a la carpeta con la copia hecha por mongodump.
- Es necesario que el usuario tenga permisos para escribir en las BDs que se quieren restaurar.

```
$ mongorestore -u <usuario> --authenticationDatabase <BD-aut>

-d <BD> <carpeta-dump>
```

Copias de seguridad

- mongodump y mongorestore necesitan que MongoDB esté en marcha para realizar/restaurar una copia de seguridad.
- Su uso tiene un impacto en el rendimiento de MongoDB.
- Una alternativa es copiar las carpetas que contienen los datos con otras herramientas (p.e. rsync)
- Desventaja: las copias ocupan más que con mongodump.
- Más información:
- https://www.mongodb.com/docs/manual/core/backups/#back-up-by- copying-underlying-data-files

Ejercicio 2

- Realizar una copia de la BD “librería” creada en el ejercicio 1.
- Utilizar el usuario con permisos “root”.
- Eliminar 2 documentos de la colección “libros”.
- Restaurar la copia de seguridad y verificar que la colección vuelve a tener los documentos eliminados.

Sharding

- Hasta ahora hemos trabajado con una única instancia de MongoDB.
- Pero…
- MongoDB es un SGDB NoSQL pensado para trabajar como un sistema multi-nodo distribuido.
- Poder escalar horizontalmente facilita su crecimiento.

Sharding

- Un shard es una instancia de MongoDB que contiene una porción de los datos.
- Ejemplo:
- 5 servidores, 1 instalación de MongoDB
- Gestionan una BD sobre libros.
- Cada Shard contiene un conjunto de los documentos.

Shard 1

Libros A a E

Shard 2

Libros F a J

Shard 3

Libros L a P

Shard 4

Libros Q a U

Sharding

- El punto de acceso a una instalación MongoDB distribuida es mongos
- Proceso que actúa como *router*.
- Recibe peticiones y las redirige al Shard adecuado.

21

Imagen de: https://www.mongodb.com/docs/manual/sharding/

Se recomienda instalarlo en los servidores que contienen la aplicación

Sharding

- Los documentos de las colecciones se distribuyen entre Shards en base a una “Shard key”.
- Es uno de los campos de los documentos, elegida por el desarrollador y/o administrador.
- En el ejemplo de la diapositiva 20, sería el título del libro.
- Debe elegirse con cuidado, ya que afecta al rendimiento del sistema.
- Limita la concurrencia.

22

Sharding

- 2 escenarios posibles en una instalación distribuida
- Consulta que no hace uso de Shard Key
- mongos debe enviar la consulta a todos los Shards.
- Mayor saturación de la red.

Imagen de: https://www.mongodb.com/docs/manual/core/sharded-cluster-query-router/

Sharding

- 2 escenarios posibles en una instalación distribuida
- Consulta que hace uso de Shard Key
- mongos envía la consulta al Shards (o Shards) que contienen la respuesta.
- Mejor tiempo de respuesta, menor uso de ancho de banda.

Imagen de: https://www.mongodb.com/docs/manual/core/sharded-cluster-query-router/

Sharding

- Cuándo considerar una instalación distribuida de MongoDB:
- Para atender consultas con muy baja latencia.
- Para gestionar conjuntos de datos muy grandes.
- Una instalación distribuida provee alta disponibilidad y alto rendimiento pero es compleja de gestionar.
- Más información:
- https://www.mongodb.com/docs/manual/sharding/

Bibliografía

- M. Schwarzmüller. “MongoDB - The Complete Developer's Guide”, Udemy, 2022.
- https://www.udemy.com/course/mongodb-the-complete- developers-guide
- MongoDB Documentation, 2022.
- https://www.mongodb.com/docs